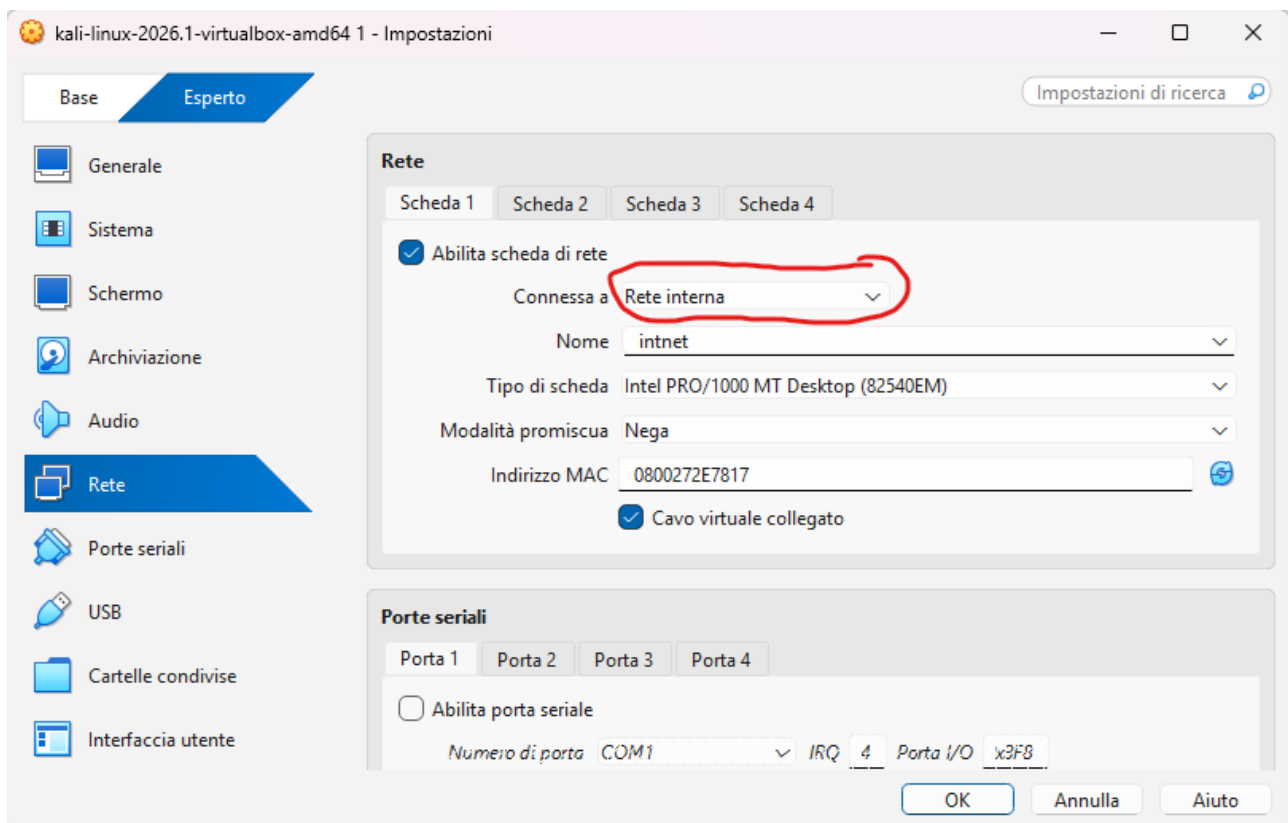


Report sulla lezione W2D4

In questa lezione abbiamo imparato a far comunicare 2 virtual machine tra di loro, ossia **Kali Linux** e **Metasploitable 2**.

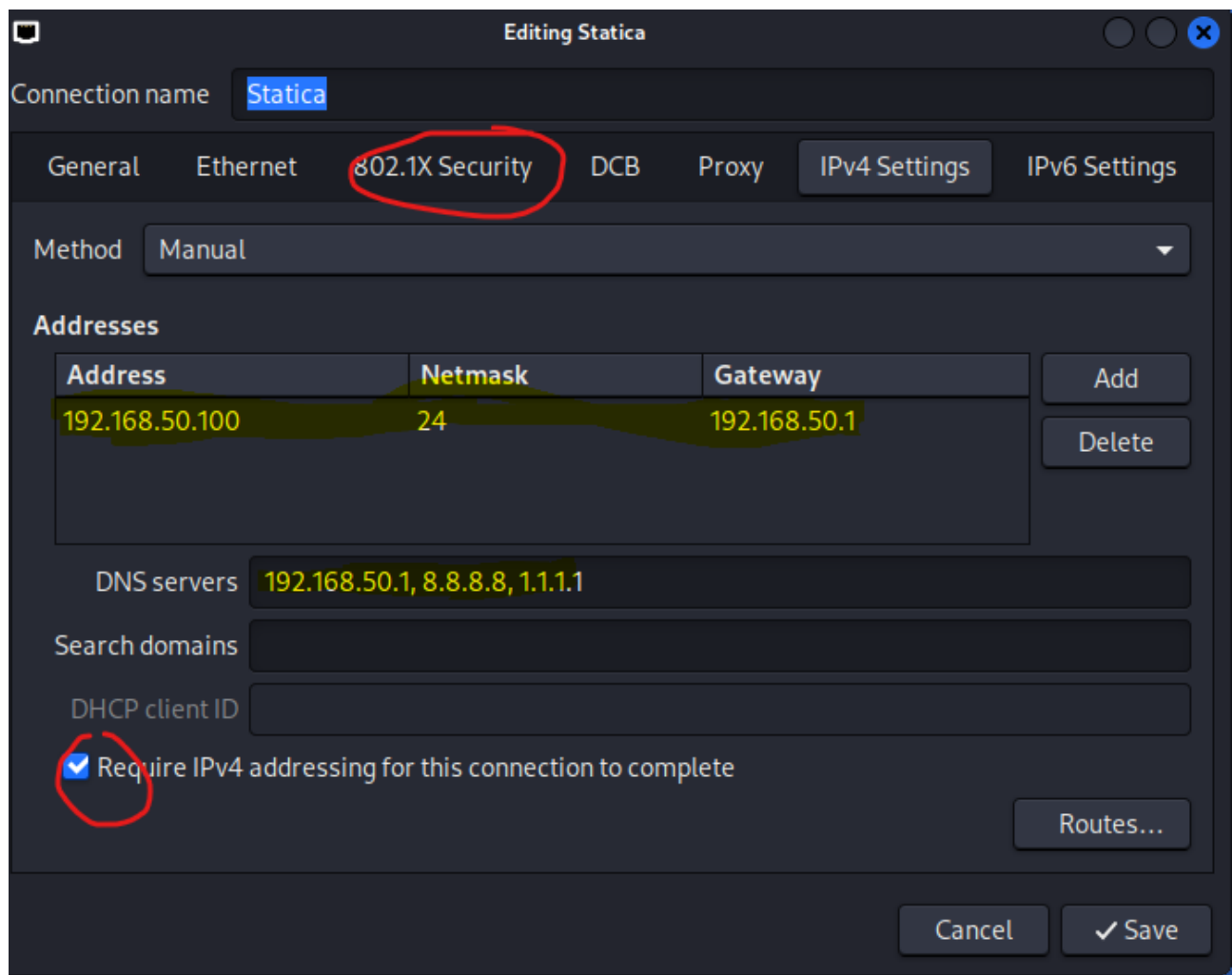
Per fare ciò ci sono molti piccoli passaggi da fare per assicurarsi che tutto funzioni come debba.

1. A macchine spente si va nelle impostazioni sia di Kali che di Meta2, si clicca sulla modalità esperto per comodità nostra e nelle impostazioni di rete modifichiamo la scheda 1, la abilitiamo e modifichiamo queste voci esattamente così:



Importante aggiungere che nella voce “**Nome**” bisogna nominare la rete sia in Kali che in Meta2 nello stesso modo, altrimenti la comunicazione non potrà avvenire.

2. Abbiamo avviato la macchina di Kali e in alto a destra nelle impostazioni di ethernet network connection con il tasto destro abbiamo configurato una rete chiamata statica con i seguenti parametri :



Ho cerchiato la parte 802.1X Security perché se attiva non lascia creare la rete così come vorremmo impostarla noi.

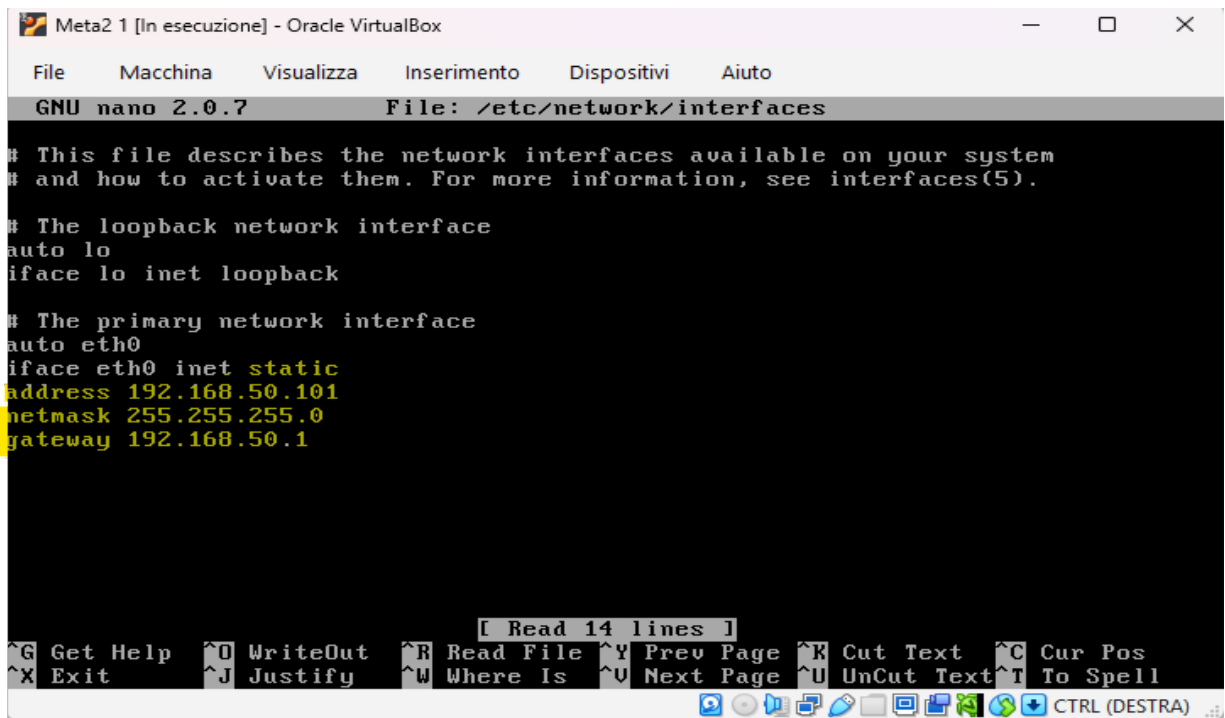
3. In seguito per la parte finale di Kali abbiamo aperto il prompt dei comandi e digitato il comando seguente comando ;” ip a” per accertarci che tutto sia andato alla perfezione, questo è il risultato:

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:2e:78:17 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.50.100/24 brd 192.168.50.255 scope global noprefixroute eth0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::42e9:45dd:3162:4903/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
(kali@kali)-[~]  
$
```

La parte che ci interessa al momento era appunto vedere se con questo comando ci prendeva l'**inet** che abbiamo immesso nelle impostazioni, in questo caso è giusto 192.168.50.100/24.

4. Passando a Meta2 invece il discorso è diverso, dopo l'avvio abbiamo inserito il seguente comando per accedere alla configurazione delle reti :” **sudo nano /etc/network/interfaces**”.

Immettendo questo comando si arriva così a questa schermata, dove abbiamo impostato i seguenti valori :



```
Meta2 1 [In esecuzione] - Oracle VirtualBox
File  Macchina  Visualizza  Inserimento  Dispositivi  Aiuto
GNU nano 2.0.7  File: /etc/network/interfaces

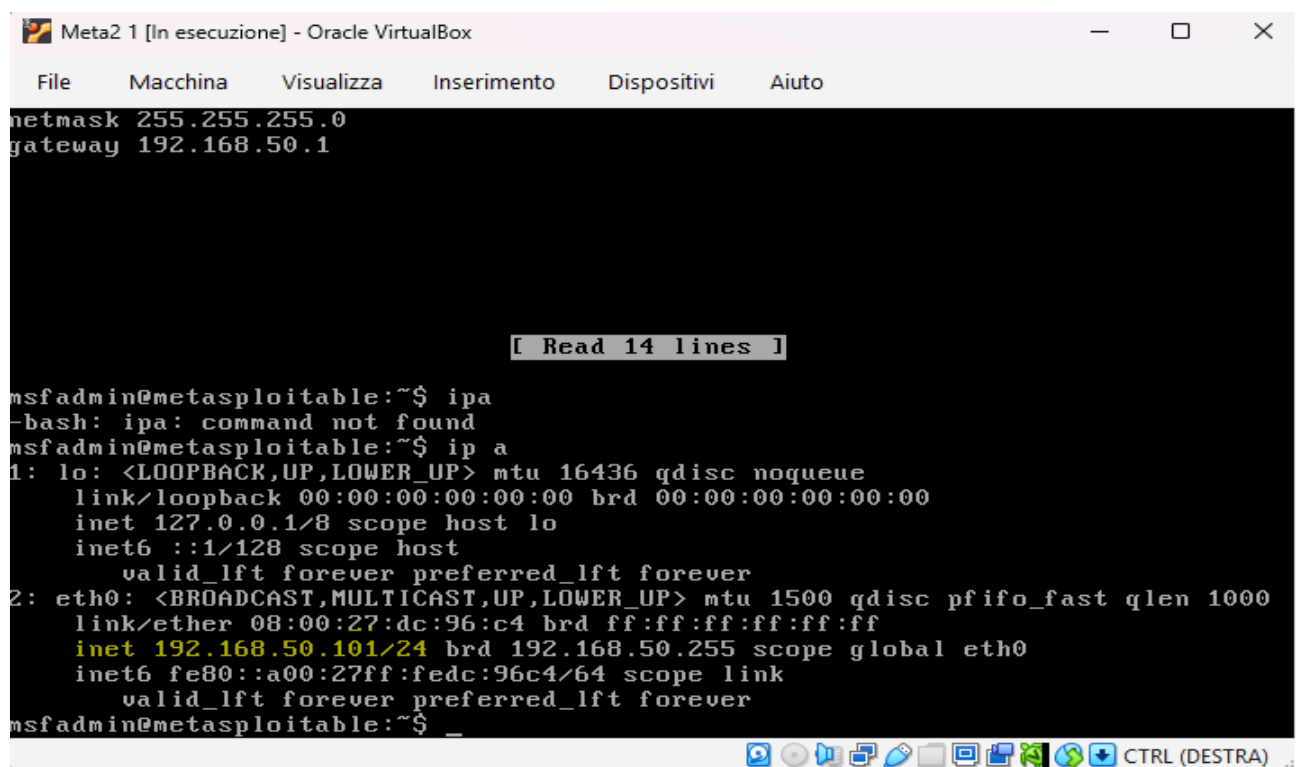
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto eth0
iface eth0 inet static
address 192.168.50.101
netmask 255.255.255.0
gateway 192.168.50.1

[ Read 14 lines ]
^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Page  ^K Cut Text    ^C Cur Pos
^X Exit      ^J Justify    ^W Where Is   ^V Next Page  ^U UnCut Text ^T To Spell
CTRL (DESTRA)
```

Dove è evidenziato static prima c'era l'opzione DHCP che è automatica, mentre dove troviamo address/netmask/gateway era vuoto e abbiamo immesso le informazioni a Meta2 in modo manuale. Infine abbiamo salvato con **CTRL+X** e poi abbiamo dato lo stesso comando "ip a" per verificare che fosse tutto apposto:



```
Meta2 1 [In esecuzione] - Oracle VirtualBox
File  Macchina  Visualizza  Inserimento  Dispositivi  Aiuto

netmask 255.255.255.0
gateway 192.168.50.1

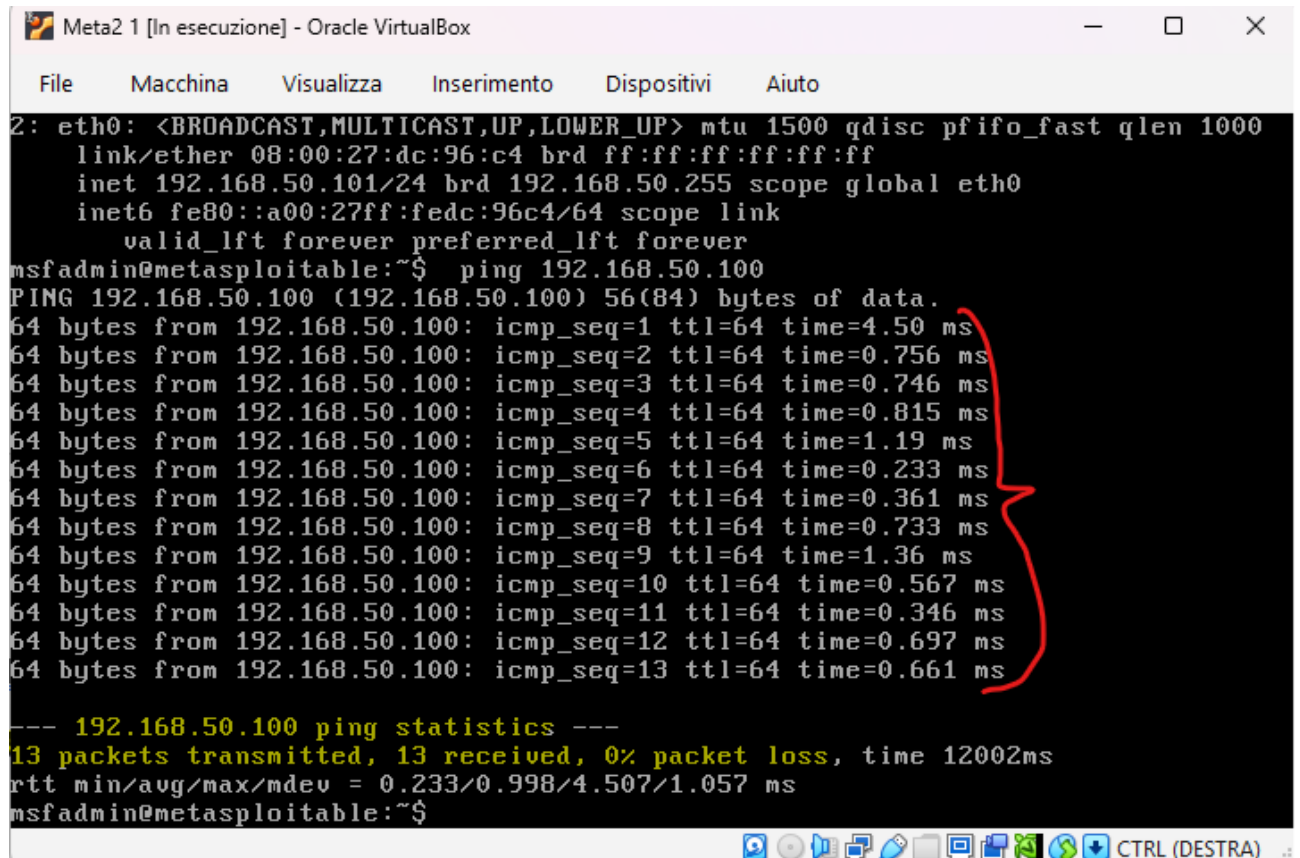
[ Read 14 lines ]

msfadmin@metasploitable:~$ ipa
-bash: ipa: command not found
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:dc:96:c4 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.101/24 brd 192.168.50.255 scope global eth0
    inet6 fe80::a00:27ff:fedc:96c4/64 scope link
        valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$ _

CTRL (DESTRA)
```

Infatti l'inet nel mio caso è 192.168.50.101/24.

5. Nel prossimo passaggio partendo da Meta 2 abbiamo provato a fare un ping test a Kali e come si osserva nello screenshot i pacchetti continuano ad essere inviati, segno che il ping sta funzionando :



```
Meta2 1 [In esecuzione] - Oracle VirtualBox
File  Macchina  Visualizza  Inserimento  Dispositivi  Aiuto

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:dc:96:c4 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.101/24 brd 192.168.50.255 scope global eth0
    inet6 fe80::a00:27ff:fedc:96c4/64 scope link
        valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$ ping 192.168.50.100
PING 192.168.50.100 (192.168.50.100) 56(84) bytes of data:
64 bytes from 192.168.50.100: icmp_seq=1 ttl=64 time=4.50 ms
64 bytes from 192.168.50.100: icmp_seq=2 ttl=64 time=0.756 ms
64 bytes from 192.168.50.100: icmp_seq=3 ttl=64 time=0.746 ms
64 bytes from 192.168.50.100: icmp_seq=4 ttl=64 time=0.815 ms
64 bytes from 192.168.50.100: icmp_seq=5 ttl=64 time=1.19 ms
64 bytes from 192.168.50.100: icmp_seq=6 ttl=64 time=0.233 ms
64 bytes from 192.168.50.100: icmp_seq=7 ttl=64 time=0.361 ms
64 bytes from 192.168.50.100: icmp_seq=8 ttl=64 time=0.733 ms
64 bytes from 192.168.50.100: icmp_seq=9 ttl=64 time=1.36 ms
64 bytes from 192.168.50.100: icmp_seq=10 ttl=64 time=0.567 ms
64 bytes from 192.168.50.100: icmp_seq=11 ttl=64 time=0.346 ms
64 bytes from 192.168.50.100: icmp_seq=12 ttl=64 time=0.697 ms
64 bytes from 192.168.50.100: icmp_seq=13 ttl=64 time=0.661 ms

--- 192.168.50.100 ping statistics ---
13 packets transmitted, 13 received, 0% packet loss, time 12002ms
rtt min/avg/max/mdev = 0.233/0.998/4.507/1.057 ms
msfadmin@metasploitable:~$
```

13 pacchetti trasmessi e 13 ricevuti, 0% dei pacchetti persi, funziona tutto alla perfezione.

6. Infine, dalla Kali abbiamo provato a fare lo stesso test per verificare che funzioni :

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ ping 192.168.50.101  
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.  
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=5.44 ms  
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=0.984 ms  
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=0.443 ms  
64 bytes from 192.168.50.101: icmp_seq=4 ttl=64 time=0.736 ms  
64 bytes from 192.168.50.101: icmp_seq=5 ttl=64 time=0.568 ms  
64 bytes from 192.168.50.101: icmp_seq=6 ttl=64 time=0.792 ms  
64 bytes from 192.168.50.101: icmp_seq=7 ttl=64 time=0.785 ms  
^C  
— 192.168.50.101 ping statistics —  
7 packets transmitted, 7 received, 0% packet loss, time 6107ms  
rtt min/avg/max/mdev = 0.443/1.392/5.442/1.660 ms  
(kali@kali)-[~]  
$
```

Anche in questo caso nessun pacchetto perso quindi tutto ha funzionato come doveva.